

CSCE 711 APPLIED CRYPTOGRAPHY · MODULE 13 · LECTURE NOTES

Claiming Cryptographic Security Honestly

Which properties, against whom, assuming what.

MODULE OUTCOMES

- State a cryptographic claim completely.
- Identify the standard overclaims.
- Explain what a security proof does and does not establish.
- Place this course relative to CSCE 701 and CSCE 713.
- State the course's rule in your own words.

1 The complete claim

A COMPLETE CLAIM NAMES THE PROPERTY, THE ADVERSARY, THE ASSUMPTION, AND THE SCOPE

The property: which of Module 01 §1's four, specifically — confidentiality, integrity, authentication, or freshness, named individually because a system may provide some and not others. **Not 'secure', which names none of them.**

The adversary: a passive network observer, an active network attacker, a compromised server, a malicious client, the cloud operator, a legal process, an insider — because the answer is genuinely different for each, and a claim that does not say which is not falsifiable.

The assumption: which hardness assumption the scheme rests on, and what key management is being presumed correct (Module 06 §2, Module 10) — the second being the one always left implicit and most often wrong.

And the scope: which data, at which points in its lifetime — in transit, at rest, in use in process memory, in backups, in logs, in the analytics pipeline. **'Encrypted at rest' says nothing about the other five.**

The claim	The correction
'It is encrypted.'	Which property? Against which adversary? With the keys held where and readable by whom? The sentence conveys almost nothing.
'Military-grade encryption.'	Meaningless as a phrase, and misdirected as a claim — the primitive was never the weak point (Module 01 §4).
'256-bit encryption.'	A key length, not a security property (Module 03 §1) — see the note.

The claim	The correction
'End-to-end encrypted.'	Incomplete without the recovery mechanism , which is the weakest point of the design (Module 10 §4).
'Provably secure.'	Proved relative to a stated assumption, inside a stated model — and almost every real break was outside the model (§3).
'Quantum-safe.'	Only if actually deployed, and hybrid is the honest form (Module 12 §4).
'Zero-knowledge.'	A specific technical property with a formal definition, not a marketing synonym for 'we cannot see your data'.

'256-bit encryption' is the most revealing of these. It describes a parameter and implies a guarantee — and it is perfectly compatible with ECB mode, a hardcoded key, no authentication, and a nonce that repeats. **Which is exactly the substitution this module exists to prevent:** quoting a number that sounds large in place of stating a property that can be checked.

2 Claims you can defend

- 'Message contents are confidential and authenticated against a network observer, using ChaCha20-Poly1305 with keys exchanged by X25519, assuming the discrete logarithm problem is hard against a classical adversary.' Property, adversary, construction, assumption.
- 'The server can read message contents. This is not an end-to-end encrypted system.' Which is an honest statement of an architecture rather than an admission (Module 10 §4) — and is far more useful to a user than a vague reassurance.
- 'Data at rest is encrypted with per-object data keys under a master key held in a key service. A compromise of the application process exposes the objects that process decrypts.' Scope, construction, and blast radius.
- 'Passwords are stored using Argon2id at these specific parameters, which resists offline attack at approximately this rate on current hardware.' A measured capability rather than an assertion (Module 09 §2).
- And: 'we do not protect metadata — who communicates with whom, when, and how much is visible to us and to a network observer.' Stated rather than omitted (Module 01 §2). Every one of these is longer than the overclaim it replaces, and every one is checkable by somebody else — which is precisely the trade this module is asking you to make.

3 What a proof means

A SECURITY PROOF IS A REDUCTION, AND IT IS ONLY AS GOOD AS ITS ASSUMPTIONS AND ITS MODEL

What it establishes: if an adversary breaks this scheme, then that adversary can be used to solve the underlying hard problem — so the scheme is at least as hard to break as the problem is to solve. A reduction, exactly in CSCE 637's sense.

Which is genuinely valuable and should not be undersold: it rules out entire classes of attack at once, and replaces 'nobody has managed to break it yet' with a structural argument that will remain true as long as the assumption holds.

What it does not cover: the implementation (Module 11), the key management (Module 10), side channels of any kind, the protocol the scheme is embedded in, the error handling around it, and anything at all outside the model's definition. A proof in the random oracle model assumes something no real hash function is.

So 'provably secure' is not a claim about a system. It is a claim about a scheme inside a model — and nearly every real-world break happened outside the model, in exactly the places the proof said nothing about. Which is Module 01 §4's historical record and this module's thesis, meeting.

4 The semester, and where this leaves you

Course	What it covers	The honest summary
CSCE 701	Systems, operations, architecture, and people.	Where the incidents are — credentials, misconfiguration, missing authorisation, unpatched dependencies.
CSCE 711 (this one)	The primitives, and their correct use and composition.	The part that works — misused. See the note.
CSCE 713	Finding and preventing defects in code.	Where the severe bugs are, and where CSCE 627's undecidability bounds what tooling can promise.

This course's honest summary is the uncomfortable one: the mathematics is sound, and the failures are yours — reused nonces, keys in repositories, distinguishable errors, unauthenticated encryption, keys serving two purposes, and comparisons that are not constant-time. **Which is why this course spent more time on Modules 10 and 11 than on the primitives themselves**, and why Module 01 §4's rule was stated at the beginning rather than derived at the end.

WHERE THIS COURSE LEAVES YOU

You can state what each primitive provides, choose correctly from a reviewed library, and recognise the compositions that are unsound — which is the practical skill, and **it transfers to systems you did not design and cannot rewrite**, which is most of them.

You can design a key lifecycle, specify envelope encryption, and execute a rotation you have actually tested — which is the subject that genuinely decides whether any of the cryptography helps, and which is the half most courses leave out.

You can find the implementation hazards: a non-constant-time comparison, a nonce whose uniqueness does not survive a restart, a distinguishable decryption error, a key serving two purposes, a parser running before verification. Five checks, mechanically applicable.

And the closing rule is the program's, unchanged across twenty-six courses: state what you measured, state what you assumed, and never claim more than you established. In cryptography it means naming the property, the adversary, and the assumption — because **'it is encrypted'** assumes **all three and states none of them**, and the whole of §1's table is what happens when that substitution goes unchallenged.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Boneh & Shoup, the introduction and chapter 2 (free)	§3's account of what a proof establishes , from the people who write them — and notably careful about the limits.
Koblitz & Menezes — Another Look at “Provable Security” (free)	§3's caveats argued forcefully — a useful corrective, and contested, which is itself instructive.
Green — A Few Thoughts on Cryptographic Engineering (free)	§1 and §2 in practice — consistently good at separating what a system claims from what it provides.
Anderson — Security Engineering, the cryptography chapter (free PDF)	§4's placement of this subject within security as a whole , and consistent with this course's ordering.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Take a cryptographic claim made publicly** by a product you use and assess it against §1's table.
- 2 Rewrite it** in the four-component form.

- 3 **Write the complete claim for your own system** — property, adversary, assumption, scope.
- 4 **State what you do not protect**, explicitly.
- 5 **Find a 'provably secure' claim** and identify what its model excludes.
- 6 **List the places a break could occur outside the model** for one scheme you use.
- 7 **Compare your actual effort allocation** against the three courses' failure sources.
- 8 **Revisit Module 01's last exercise** and compare what you wrote then.
- 9 **Run Module 11's five checks** once more, on your own project.
- 10 **Project 2 is now due.** Submit the requirement, the primitive choices with justification, the complete key lifecycle, the executed rotation, and the explicit list of properties you are not providing.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name the four components of a complete claim.
- 2 Why does the adversary have to be named?
- 3 Give seven overclaims and the correction to each.
- 4 Why is '256-bit encryption' the most revealing?
- 5 Give three claims you could actually defend.
- 6 What does a security proof establish, and as a what?
- 7 What does a proof not cover — give five things.
- 8 Why is 'provably secure' not a claim about a system?
- 9 How do the three security courses divide?
- 10 In this course, what does 'never claim more than you established' mean?